

Điều khiển giao dịch
đồng thời

Nội dung

- Kỹ thuật khóa.
 - Khóa nhị phân
 - Khóa đọc/ghi
- Khóa 2 pha.
- Deadlock và Starvation.
 - Deadlock Prevention.
 - Deadlock Detection.
- Kỹ thuật nhãn thời gian.
- Kỹ thuật sử dụng nhiều phiên bản.

Giới thiệu

- Tìm hiểu một số kỹ thuật điều khiển song hành (Concurrency control) được sử dụng trong việc đảm bảo tính cô lập của các giao dịch được thực hiện.
- Các kỹ thuật này đảm bảo tính khả tuần tự của lịch trình dựa trên các **giao thức điều khiển song hành – Concurrency control protocols** (protocols – sets of rules)

Khái niệm

KĨ THUẬT KHÓA

Giao thức dựa trên khóa

- Một phương pháp để đảm bảo tính tuần tự dựa trên khái niệm khóa (LOCKING) các hạng mục dữ liệu
- Kỹ thuật khóa ngăn chặn nhiều giao dịch truy xuất 1 hạng mục dữ liệu trong cùng 1 thời điểm => giảm khả năng làm giả dl hoặc chỉnh sửa
- Cơ chế khóa được sử dụng trong hầu hết các hệ quản trị CSDL thương mại.
- Yêu cầu việc truy xuất đến một hạng mục dữ liệu được tiến hành theo kiểu loại trừ lẫn nhau (mutual exclusion).
- Một giao dịch đang truy xuất 1 hạng mục dữ liệu thì **không** cho phép giao dịch khác chỉnh sửa dữ liệu này.

Giao thức dựa trên khóa

là 1 biến bool

- Một khóa (lock) là một biến tương ứng với một hạng mục dữ liệu, quy định những hành động cụ thể nào được phép thực hiện trên hạng mục dữ liệu đó.
- Thông thường: 1 khóa cho mỗi hạng mục dữ liệu.
- Có nhiều loại khóa được sử dụng trong điều khiển song hành.

song hành là chạy xen kẽ
song song là chạy cùng lúc

Khóa nhị phân

KĨ THUẬT KHÓA

Khóa nhị phân

- Đơn giản nhưng rất hạn chế nên không dùng trong thực tế
- 1 khóa nhị phân (binary lock) gồm 2 trạng thái:
 - Locked (1)
 - Unlocked (0)  ví dụ hạng mục X sẽ có biến khóa $LX = 1$ hoặc Y sẽ có LY để khóa
- Các khóa khác nhau trên mỗi hạng mục dữ liệu khác nhau.
- Nếu trạng thái khóa của X là 1, hạng mục dữ liệu X không thể được truy xuất bởi các thao tác dữ liệu khác.
 - $Lock(X) = 1$

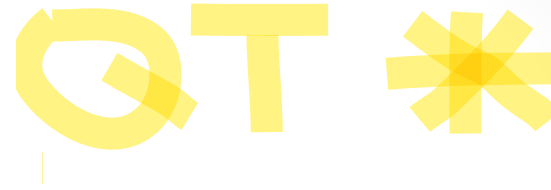
Khóa nhị phân

- 2 thao tác trong khóa nhị phân:
 - $\text{Lock_item}(X)$ $\Rightarrow$ xin phép được khóa $X \Rightarrow$ phải kiểm tra X có Unlock không nếu có mới được khóa
 - $\text{Unlock_item}(X)$ $\Rightarrow$ nếu mở khóa sẽ có khả năng bị chiếm hữu bởi một giao dịch khác
- Khi một giao dịch muốn truy xuất X , trước tiên nó thực hiện một thao tác $\text{Lock_item}(X)$.
 - Nếu $\text{Lock}(X) = 1$, giao dịch phải đợi.
 - Nếu $\text{Lock}(X) = 0$, giá trị $\text{Lock}(X)$ gán thành 1, giao dịch được thao tác trên X .

Khóa nhị phân

- Sau khi hoàn tất những thao tác trên X, giao dịch thực hiện thao tác `Unlock_item(X)`:
 - Gán `Lock(X)=0`
- Khi đó, X có thể được truy xuất bởi các giao dịch khác.
- Giai đoạn giữa `Lock_item(X)` và `Unlock_item(X)`, giao dịch được gọi là đang giữ khóa trên X.
- Chỉ một giao dịch được giữ khóa trên 1 hạng mục dữ liệu.

Khóa nhị phân



- Thao tác khóa/mở khóa:

```
lock_item(X):  
  B:  if LOCK(X) = 0          (* item is unlocked *)  
        then LOCK(X) ← 1      (* lock the item *)  
      else  
        begin  
          wait (until LOCK(X) = 0  
                and the lock manager wakes up the transaction);  
          go to B  
        end;  
unlock_item(X):  
  LOCK(X) ← 0;                (* unlock the item *)  
  if any transactions are waiting  
    then wakeup one of the waiting transactions;
```

Khóa nhị phân

- Khóa nhị phân được thực hiện đơn giản bằng cách thêm 1 biến nhị phân ứng với mỗi hạng mục dữ liệu.
- Mỗi khóa được ghi nhận với 3 thuộc tính cơ bản:
<Data_item_name, LOCK, Locking_transaction>
 - Tên hạng mục dữ liệu.
 - Giá trị khóa LOCK
 - Giao dịch giữ khóa
- Hệ thống quản lý các khóa trong một bảng, các hạng mục dữ liệu không có trong bảng là không khóa (unlocked).

item	lock	trans

Khóa nhị phân → ĐỘC QUYỀN

Khi đó mỗi giao dịch phải tuân theo các luật:

1. Một giao dịch phải thực hiện thao tác Lock_item(X) trước khi thực hiện bất kì thao tác đọc/ghi nào trên X.
2. Một giao dịch phải Unlock_item(X) sau khi hoàn tất tất cả thao tác đọc/ghi trên X.
3. Một giao dịch không thực hiện thao tác Lock_item(X) nếu nó đang giữ khóa trên X.
4. Một giao dịch không thực hiện thao tác Unlock_item(X) trừ khi nó đang giữ khóa trên X.

Khóa đọc/ghi

KĨ THUẬT KHÓA

Khóa Shared/Exclusive

↳ GIỐNG NHỊ PHÂN

- Khóa **nhị phân** hạn chế khi dùng cho dữ liệu vì **chỉ một giao dịch được giữ khóa** trên một hạng mục dữ liệu X.
- Các thao tác đọc cùng hạng mục dữ liệu trên các giao dịch khác nhau là không xung đột → Ta có thể cho phép nhiều giao dịch được truy xuất X nếu các giao dịch đó chỉ đọc X.
- **Nếu một giao dịch** thực hiện việc **ghi lên X**, nó được **cấp quyền truy xuất riêng (exclusive)**.
- **Shared locks** hay còn gọi là **read locks**: các giao dịch khác được phép đọc. *các giao dịch được cung cấp khóa*
- **Exclusive locks** hay còn gọi là **write locks**: chỉ **1 giao dịch giữ khóa được thao tác**.

SHARE xảy ra khi cùng read không chỉnh sửa (write)

Khóa Shared/Exclusive

- Có 3 thao tác trong giao thức khóa này:
 - Read_lock(X)
 - Write_lock(X)
 - Unlock(X)
- Tương tự, giá trị khóa trên X – Lock(X) có 3 trạng thái:
 - Read-locked (Shared-lock)
 - Write-locked (Exclusive-lock)
 - Unlocked

Khóa Shared/Exclusive

- Một phương pháp thực thi giao thức khóa này là lưu các giao dịch giữ khóa trong một bảng.
- Mỗi dòng trong bảng có 4 thông tin:
 - Tên hạng mục dữ liệu
 - Giá trị của khóa: read-locked/write-locked
 - Số lượng giao dịch đọc
 - Các giao dịch giữ khóa

name	lock	Qty	list trans
X	R_L	1	T1
Y	W_L	1	T2
Z	R_L	3	T1,T2,T3
Y	R_L	2	T1,T3
Z	R_L	1	T4

exclusive:
độc quyền nên
không được
khóa đọc

Chỉ xảy ra khi Y đã được viết
xong (tức đã được unlock)

Khóa Shared/Exclusive

Hệ thống phải tuân theo các luật:

1. Một giao dịch T phải thực hiện thao tác Read_lock(X) hoặc Write_lock(X) trước khi đọc X.
2. T phải thực hiện thao tác Write_lock(X) trước khi ghi X.
3. T phải thực hiện thao tác Unlock(X) sau khi hoàn tất việc đọc, ghi trong T.

Khóa Shared/Exclusive

4. T không thực hiện $\text{Read_lock}(X)$ nếu nó đang giữ khóa read hoặc write trên X.
5. T không thực hiện $\text{Write_lock}(X)$ nếu nó đang giữ khóa read hoặc write trên X.
6. T không thực hiện $\text{Unlock}(X)$ trừ khi nó đang giữ khóa read hoặc write trên X.

Thao tác khóa đọc

read_lock(X):

```
B:  if LOCK( $X$ ) = "unlocked"
      then begin LOCK( $X$ )  $\leftarrow$  "read-locked";
           no_of_reads( $X$ )  $\leftarrow$  1
           end
      else if LOCK( $X$ ) = "read-locked"
           then no_of_reads( $X$ )  $\leftarrow$  no_of_reads( $X$ ) + 1
      else begin
           wait (until LOCK( $X$ ) = "unlocked"
                and the lock manager wakes up the transaction);
           go to B
           end;
```

Thao tác khóa ghi

tìm hiểu thêm có thêm chức năng có timer

```
write_lock(X):  
B:  if LOCK(X) = "unlocked"  
    then LOCK(X) ← "write-locked"  
    else begin  
        wait (until LOCK(X) = "unlocked"  
              and the lock manager wakes up the transaction);  
        go to B  
    end;
```

Thao tác mở khóa

```
unlock (X):  
  if LOCK(X) = "write-locked"  
    then begin LOCK(X) ← "unlocked";  
             wakeup one of the waiting transactions, if any  
    end  
  else if LOCK(X) = "read-locked"  
    then begin  
           no_of_reads(X) ← no_of_reads(X) - 1;  
           if no_of_reads(X) = 0  
             then begin LOCK(X) = "unlocked";  
                      wakeup one of the waiting transactions, if any  
            end  
    end  
  end;
```

Khóa Shared/Exclusive

Chuyển đổi khóa (Lock conversion):

- Nâng cấp (Upgrade) khóa: Nếu giao dịch T là giao dịch **duy nhất** đang giữ khóa read trên X tại thời điểm thực hiện thao tác Write_lock(X), khóa read này sẽ được nâng cấp thành khóa Write, nếu không giao dịch phải đợi.
- Hạ cấp (Downgrade) khóa: từ khóa Write chuyển thành khóa read.
- Khi đó bảng lưu thông tin về khóa phải thay đổi giá trị tương ứng.

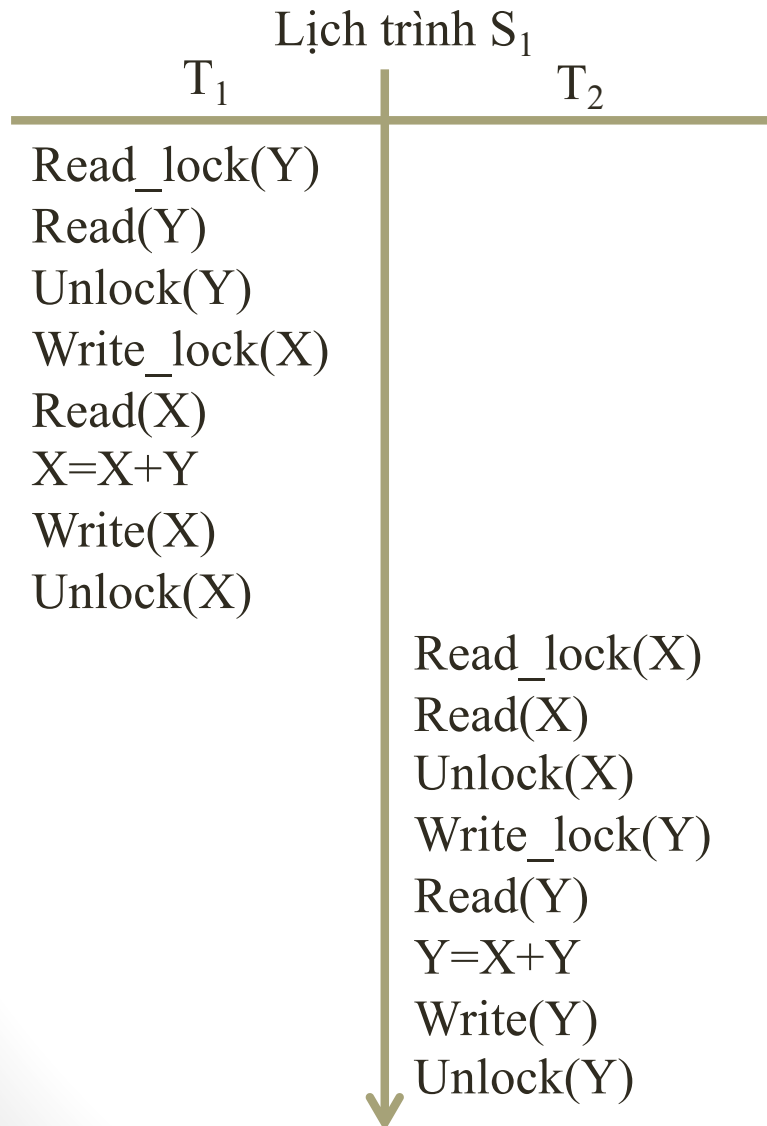
Giới thiệu

KHÓA 2 PHA

Kĩ thuật khóa 2 pha

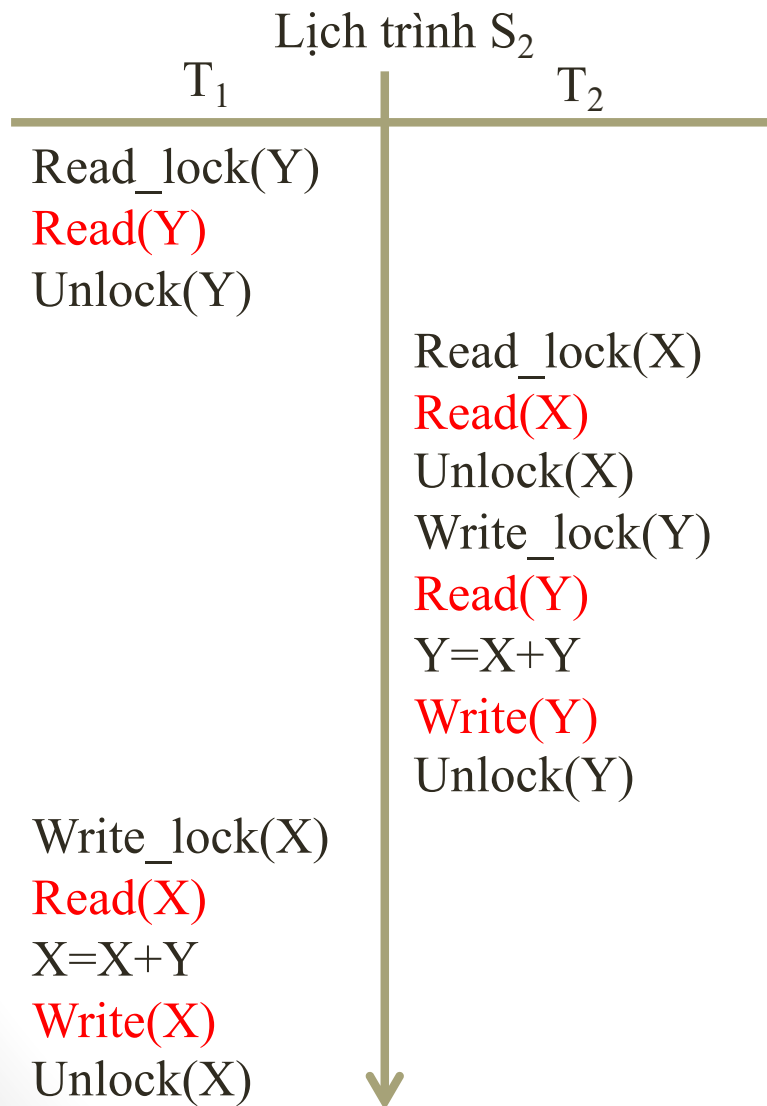
- Về mặt lý thuyết ta thấy rằng kĩ thuật khóa sẽ đảm bảo được tính khả tuần tự.
- Tuy nhiên trên thực tế, khi thực hiện đúng luật của cơ chế khóa vẫn không đảm bảo hoàn toàn tính khả tuần tự khi điều khiển song hành.

Kĩ thuật khóa 2 pha



- $X=20, Y=30$
- $T1 \rightarrow T2: X=50, Y=80$
- $T2 \rightarrow T1: X=70, Y=50$

Kĩ thuật khóa 2 pha



- X=???
- Y=???
- Có đảm bảo tính nhất quán của CSDL?
- Khả tuần tự khi sử dụng cơ chế lock?

Kĩ thuật khóa 2 pha

- Nguyên nhân: Y trong T_1 và X trong T_2 được unlock quá sớm.
- ➔ **Thời điểm mở khóa** là một yếu tố quan trọng.
- Do đó để đảm bảo tính khả tuần tự phải áp dụng thêm một số giao thức khác liên quan đến thời điểm khóa và mở khóa cho mỗi giao dịch.
- Giao thức phổ biến nhất là khóa 2 pha (Two-phase locking)

Kỹ thuật khóa 2 pha

T_1	T_2
<pre>read_lock(Y); read_item(Y); unlock(Y); write_lock(X); read_item(X); X := X + Y; write_item(X); unlock(X);</pre>	<pre>read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); Y := X + Y; write_item(Y); unlock(Y);</pre>

T_1'	T_2'
<pre>read_lock(Y); read_item(Y); write_lock(X); unlock(Y); read_item(X); X := X + Y; write_item(X); unlock(X);</pre>	<pre>read_lock(X); read_item(X); write_lock(Y); unlock(X); read_item(Y); Y := X + Y; write_item(Y); unlock(Y);</pre>

Khái niệm

KHÓA 2 PHA

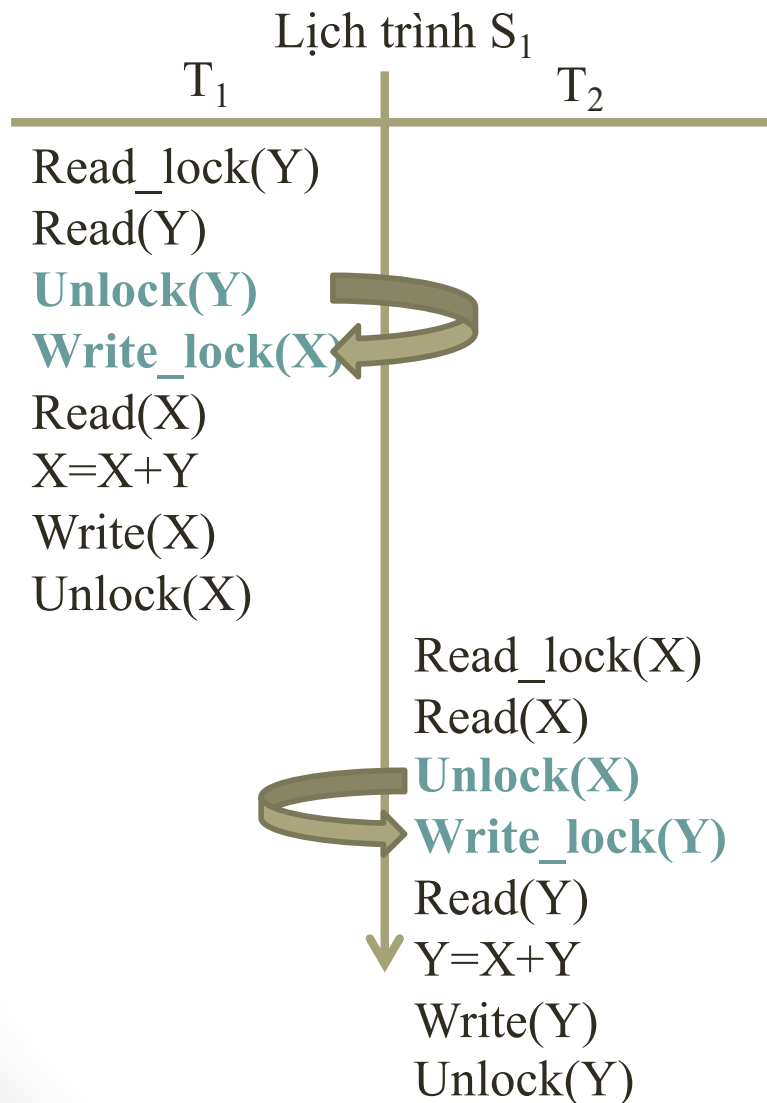
Kĩ thuật khóa 2 pha

- Tất cả thao tác khóa phải diễn ra trước thao tác mở khóa đầu tiên trong giao dịch.
- Khi đó một giao dịch được chia làm 2 pha (giai đoạn):
 - Giai đoạn đầu tiên - tạo khóa (expanding/growing phase): các khóa được phép tạo nhưng không khóa nào được phép mở.
 - Giai đoạn mở khóa (shrinking phase): các khóa đang tồn tại được phép mở nhưng không khóa mới nào được tạo.

Kỹ thuật khóa 2 pha

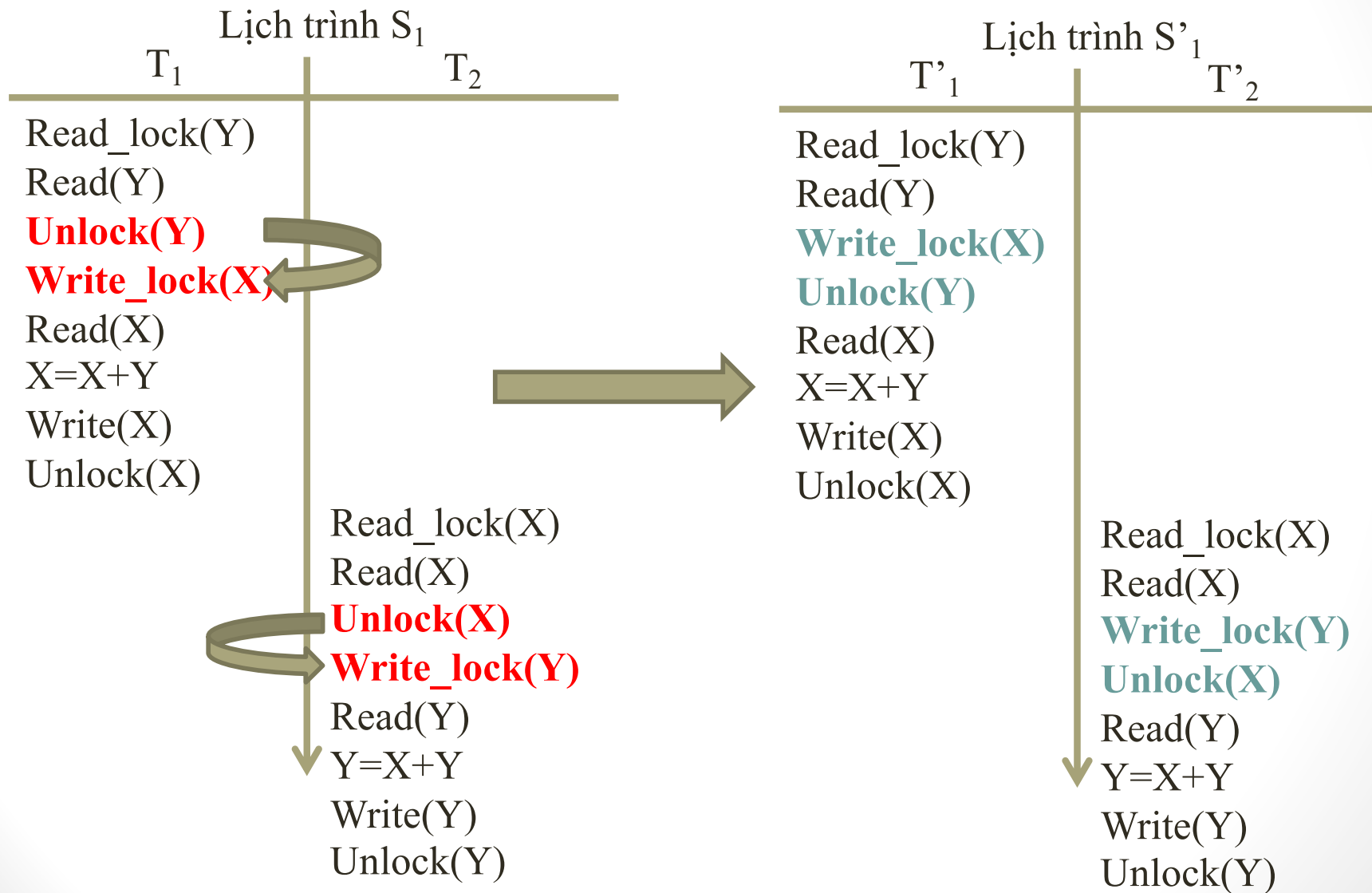
- Trong trường hợp cho phép chuyển đổi khóa:
 - Nâng cấp khóa (Read-locked $\rightarrow$ Write-locked): thực hiện ở giai đoạn tạo khóa.
 - Hạ cấp khóa: giai đoạn mở khóa.

Kĩ thuật khóa 2 pha

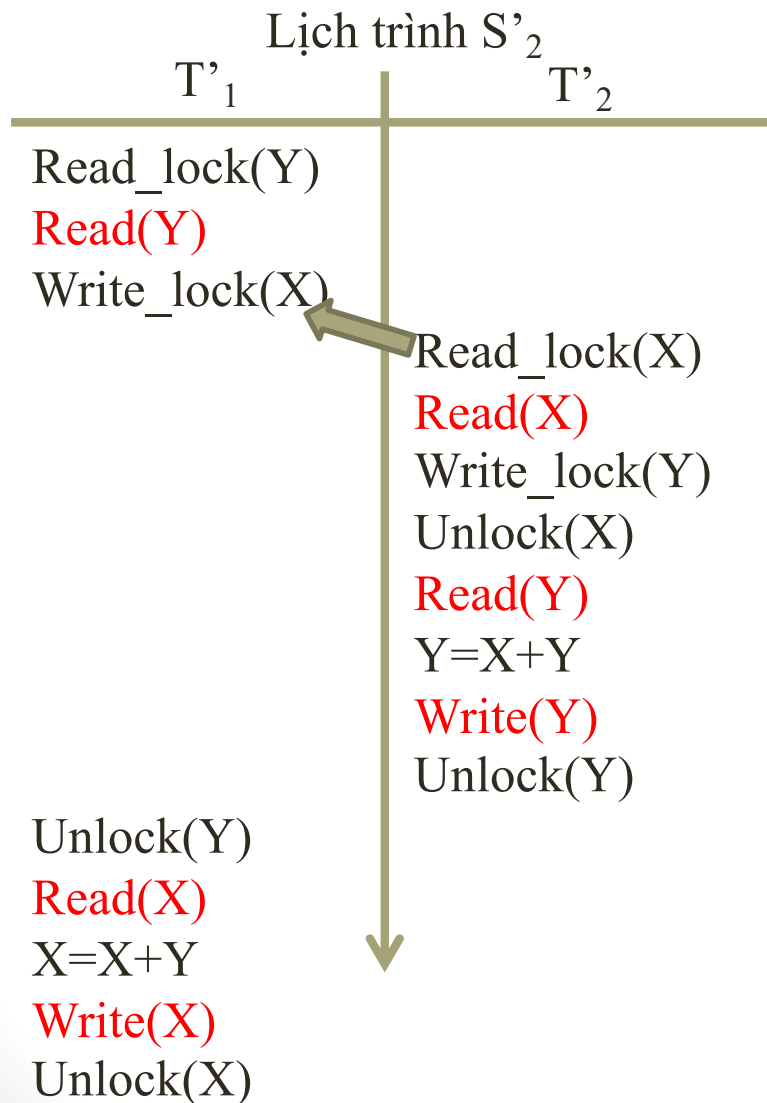


- T_1 và T_2 không theo giao thức khóa 2 pha.

Kĩ thuật khóa 2 pha



Kĩ thuật khóa 2 pha



- Read_lock(X) trong T'_2 buộc phải đợi cho đến khi T'_1 Unlock(X).
- Đảm bảo được tính khả tuần tự.

Một số hạn chế

KHÓA 2 PHA

Một số hạn chế

- Khóa 2 pha có thể giới hạn số lượng giao dịch đồng thời.
- Giao dịch T không thể mở khóa X sau khi X được sử dụng nếu sau đó T phải khóa Y
- ➔ Các giao dịch khác sử dụng đến X buộc phải đợi mặc dù có thể T đã hoàn tất thao tác trên X.

```
Read_lock(X)
Read(X)
Write_lock(Y)
Unlock(X)
Read(Y)
Y=X+Y
Write(Y)
Unlock(Y)
```

Một số hạn chế

- Hoặc là T phải khóa thêm Y mặc dù T chưa sử dụng đến Y để mở được khóa X.
- ➔ Các giao dịch khác buộc phải đợi khi thao tác trên Y mặc dù Y chưa sử dụng đến.

```
Read_lock(X)
Write_lock(Y)
Read(X)
Unlock(X)
Read(Y)
Y=X+Y
Write(Y)
Unlock(Y)
```

- Đó là cái giá của việc đảm bảo tính khả tuần tự mà không cần kiểm tra mỗi lịch trình.

Một số hạn chế

- Khóa 2 pha mặc dù đảm bảo được tính khả tuần tự nhưng nó không cho phép tất cả các lịch trình khả tuần tự do một số lịch trình bị hạn chế bởi giao thức.
- Ngoài ra còn xuất hiện 2 vấn đề:
 - Deadlock
 - Starvation

Một số biến thể khác

KHÓA 2 PHA

Một số biến thể của khóa 2 pha

- Kỹ thuật mô tả trên: Basic 2PL (2 phase locking).
- Conservative (thận trọng, bảo thủ) 2PL: giao dịch khóa toàn bộ dữ liệu cần thiết trước khi giao dịch bắt đầu thực thi (read_set và write_set).

Nếu có bất kì dữ liệu nào không thể khóa, giao dịch chờ cho đến khi tất cả dữ liệu được mở khóa → không thực tế.

Một số biến thể của khóa 2 pha

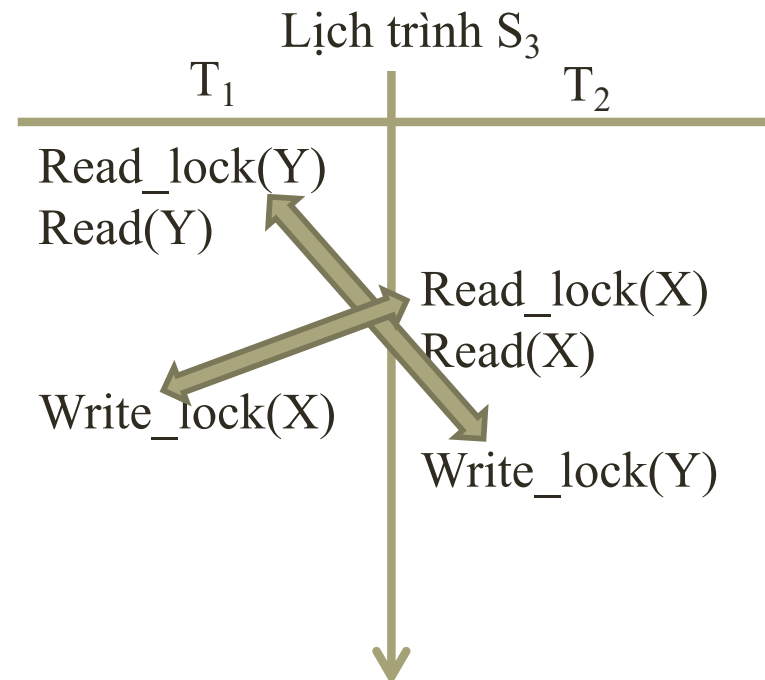
- Strict (chặt chẽ) 2PL: giao dịch không mở khóa bất kì khóa exclusive nào cho đến khi giao dịch kết thúc.
- Rigorous (khắt khe) 2PL: giao dịch không mở bất kì khóa nào cho đến khi giao dịch kết thúc.

Giới thiệu

DEADLOCK VÀ STARVATION

Deadlock

- Deadlock xảy ra khi thực hiện 2 hay nhiều giao dịch đồng thời, khi một giao dịch T chờ đợi một dữ liệu bị khóa bởi giao dịch T' và ngược lại.



Các giao thức ngăn chặn deadlock

DEADLOCK VÀ STARVATION

Deadlock Prevention Protocols

- Conservation 2PL: khóa tất cả hạng mục cần thiết trước khi thực hiện giao dịch.
- Một số kỹ thuật khác được đưa ra dựa trên việc quyết định cách thức xử lý giao dịch liên quan đến deadlock: đợi/hủy bỏ/ưu tiên, hủy giao dịch khác
- Một số trong các kỹ thuật này sử dụng khái niệm nhãn thời gian của giao dịch (Transaction timestamp)

Deadlock Prevention Protocols

- Nhãn thời gian: định danh duy nhất gán cho mỗi giao dịch dựa trên thứ tự bắt đầu của giao dịch.
 - Giao dịch bắt đầu trước (cũ hơn) sẽ có nhãn thời gian nhỏ hơn.
 - T_1 bắt đầu trước T_2 : $TS(T_1) < TS(T_2)$
- Giả sử **giao dịch T_i muốn khóa một dữ liệu X đã bị khóa bởi một giao dịch T_j khác**. Khi đó có 2 luật để ngăn chặn deadlock:
 - Wait-die
 - Wound-wait

Deadlock Prevention Protocols

Wait-die

- Nếu $TS(T_i) < TS(T_j)$:
 - T_i được phép đợi
- Ngược lại:
 - hủy T_i
 - khởi động lại T_i với $TS(T_i)$ như cũ.

Wound-wait

- Nếu $TS(T_i) < TS(T_j)$:
 - hủy T_j
 - khởi động lại T_j với $TS(T_j)$ như cũ.
- Ngược lại:
 - T_i được phép đợi

Deadlock Prevention Protocols

- Cả 2 cùng kết thúc với việc hủy bỏ giao dịch mới hơn mà **có thể** dẫn đến deadlock.
- Cả 2 kỹ thuật này đảm bảo không xuất hiện deadlock (deadlock-free) do:
 - Wait-die: chỉ đợi giao dịch bắt đầu sau (younger)
 - Wound-die: chỉ đợi giao dịch bắt đầu trước (older)
- Tuy nhiên cả 2 kỹ thuật có thể dẫn đến tình trạng hủy bỏ và khởi động lại giao dịch một cách không cần thiết trong trường hợp không xuất hiện deadlock thật sự.

Deadlock Prevention Protocols

Một số luật khác để chống deadlock không sử dụng nhãn thời gian:

- No waiting algorithm: nếu 1 giao dịch không thể khóa, nó được hủy bỏ và khởi động lại sau 1 khoảng thời gian định trước mà không cần kiểm tra deadlock.
- Cautious waiting algorithm: nếu T_j không đợi một dữ liệu bị khóa nào khác thì T_i được phép đợi; ngược lại hủy bỏ T_i .

Phát hiện deadlock

DEADLOCK VÀ STARVATION

Deadlock Detection

Wait-for Graph:

- Node: là các giao dịch đang thực hiện.
- Khi T_i đợi để khóa X đang được khóa bởi T_j : tạo cung $T_i \rightarrow T_j$.
- Khi T_j mở khóa X đó, xóa cung $T_i \rightarrow T_j$.
- Deadlock: đồ thị có chu trình.
- Lựa chọn giao dịch để hủy (victim selection): tránh các giao dịch được bắt đầu lâu, thực hiện nhiều cập nhật mà chọn giao dịch mới bắt đầu và chưa thực hiện nhiều thay đổi.

Kỹ thuật Timeouts

DEADLOCK VÀ STARVATION

Timeouts

- Một cách thức đơn giản để xử lý deadlock khác là kỹ thuật sử dụng thời gian chờ (Timeout)
- Khả thi bởi sự đơn giản.
- Timeouts: nếu một giao dịch thực hiện việc đợi trong thời gian lớn hơn một khoảng thời gian timeouts định sẵn, hệ thống xem như giao dịch gặp deadlock và hủy bỏ nó mà không cần biết deadlock có thực sự xảy ra hay không.

Starvation

DEADLOCK VÀ STARVATION

Starvation

- Khi 1 giao dịch không thể tiếp tục trong một thời gian quá lâu trong khi giao dịch khác vẫn thực hiện bình thường.
- Xảy ra do cơ chế đợi hoặc do cơ chế chọn giao dịch hủy (victim selection) không công bằng.
- Một số giải pháp:
 - First come first served
 - Chấp nhận độ ưu tiên cho các giao dịch nhưng tăng dần độ ưu tiên cho các giao dịch đợi lâu

Giới thiệu

KỸ THUẬT NHÃN THỜI GIAN

Kỹ thuật nhãn thời gian

- Kỹ thuật khóa: lịch trình tương đương với 1 vài lịch trình tuần tự (hợp với giao thức khóa)
- Kỹ thuật nhãn thời gian: lịch trình tương đương với 1 lịch trình tuần tự cụ thể được xác định dựa trên nhãn thời gian.
- Điều khiển song hành dựa trên kỹ thuật sắp xếp nhãn thời gian không sử dụng khóa do đó không phát sinh deadlock.
- Nhãn thời gian có thể phát sinh bằng nhiều cách:
 - Thứ tự tăng dần: 1, 2, 3... Tuy nhiên phải reset bộ đếm khi không có giao dịch thực hiện trong một khoảng thời gian.
 - Gán nhãn theo ngày giờ hệ thống.
- Về cơ bản, kỹ thuật này đảm bảo thứ tự truy xuất mỗi hạng mục dữ liệu bởi các chỉ thị xung đột trong lịch trình không vi phạm nhãn thời gian.

Kỹ thuật nhãn thời gian

- Để thực hiện điều này, giải thuật gắn với mỗi hạng mục dữ liệu X 2 giá trị nhãn thời gian:
 - Read_TS(X): nhãn thời gian lớn nhất trong số các nhãn thời gian của các giao dịch đọc thành công X. $\text{Read_TS}(X) = \text{TS}(T)$ với T là giao dịch mới nhất (youngest) đọc thành công X.
 - Write_TS(X): nhãn thời gian lớn nhất trong số các nhãn thời gian của các giao dịch ghi thành công X. $\text{Write_TS}(X) = \text{TS}(T)$ với T là giao dịch mới nhất (youngest) ghi thành công X.
- Có nhiều kỹ thuật nhãn thời gian khác nhau.

Kỹ thuật nhãn thời gian cơ bản
(Basic Timestamp Ordering)

KỸ THUẬT NHÃN THỜI GIAN

Thứ tự nhãn thời gian cơ bản

- Khi giao dịch T thực hiện thao tác Read(X) hoặc Write(X), giải thuật so sánh nhãn thời gian của T với Read_TS(X) hoặc Write_TS(X) để đảm bảo thứ tự thực hiện không bị vi phạm.
- Nếu thứ tự bị vi phạm, giao dịch T phải hủy và gọi lại với một nhãn thời gian mới.
- Trong tình huống T hủy bỏ và có rollback dữ liệu → giao dịch T1 nếu có sử dụng giá trị được ghi bởi T phải rollback theo.
- Tương tự nếu T2 sử dụng giá trị ghi bởi T1 cũng rollback theo.
- ➔ Cascading rollback là một vấn đề và cần có một số giao thức đi kèm

Thứ tự nhãn thời gian cơ bản

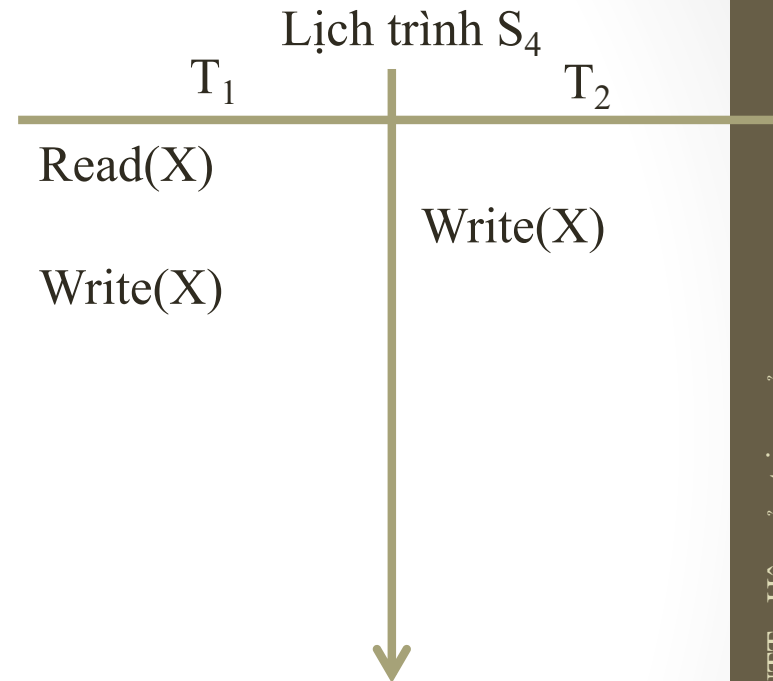
- Giao dịch T thực hiện thao tác Write(X):
 - Nếu $\text{Read_TS}(X) > \text{TS}(T)$ hoặc $\text{Write_TS}(X) > \text{TS}(T)$: hủy bỏ và rollback T vì đã có giao dịch sau T đọc hoặc ghi lên giá trị X trước khi T thực hiện Write(X).
 - Ngược lại thì thực hiện Write(X) và gán giá trị $\text{Write_TS}(X) = \text{TS}(T)$
- Giao dịch T thực hiện thao tác Read(X):
 - Nếu $\text{Write_TS}(X) > \text{TS}(T)$: hủy bỏ và rollback vì T cần đọc giá trị X cũ nhưng đã bị ghi lên bởi một giao dịch nào đó sau T.
 - Nếu $\text{Write_TS}(X) \leq \text{TS}(T)$: thực hiện Read(X) và gán $\text{Read_TS}(X)$ bằng giá trị lớn hơn trong 2 giá trị $\text{TS}(T)$ và $\text{Read_TS}(X)$.

Thứ tự nhả thời gian cơ bản

- Khi phát hiện 2 chỉ thị xung đột (xét về mặt thứ tự thực hiện của chỉ thị), giải thuật này hủy bỏ chỉ thị mới hơn bằng cách hủy bỏ giao dịch tương ứng.
- Do đó đảm bảo tính khả tuần tự.
- Deadlock không xuất hiện tuy nhiên xuất hiện việc hủy bỏ - khởi tạo lại giao dịch → có khả năng xuất hiện starvation.

Ví dụ

- $T_1 \rightarrow T_2$
- $T_1:\text{Read}(X)$ và $T_2:\text{Write}(X)$ được thực hiện thành công.
- Xét $T_1:\text{Write}(X)$
 - $\text{Write_TS}(X) ? \text{TS}(T_1)$
 - $\text{Write}(X)?? \text{Rollback}??$



Kỹ thuật nhãn thời gian chặt chẽ
(Strict Timestamp Ordering)

KỸ THUẬT NHÃN THỜI GIAN

Strict Timestamp Ordering

- Một biến thể của nhãn thời gian cơ bản đảm bảo tính khả phục hồi và tính khả tuần tự xung đột.
- Giao dịch T thực hiện thao tác $\text{Read}(X)$ hoặc $\text{Write}(X)$ mà $\text{TS}(T) > \text{Write_TS}(X)$ sẽ tạm dừng thao tác đọc/ghi đó lại cho đến khi giao dịch T' nào đó đã ghi giá trị X kết thúc (nghĩa là $\text{TS}(T') = \text{Write_TS}(X)$)

Thomas's Write Rule

KỸ THUẬT NHÃN THỜI GIAN

Thomas's Write Rule

- Một biến thể khác của kỹ thuật nhãn thời gian cơ bản không đảm bảo khả năng tuần tự xung đột của lịch trình nhưng hạn chế sự rollback trong thao tác $\text{Write}(X)$:
 - Nếu $\text{Read_TS}(X) > \text{TS}(T)$: hủy bỏ T , rollback.
 - Nếu $\text{Write_TS}(X) > \text{TS}(T)$: không thực hiện thao tác $\text{Write}(X)$ nhưng vẫn tiếp tục xử lý, **không rollback**.
 - Nếu không xảy ra 2 trường hợp trên: thực hiện thao tác $\text{Write}(X)$ và thay đổi $\text{Write_TS}(X) = \text{TS}(T)$

Giới thiệu

KỸ THUẬT NHIỀU PHIÊN BẢN

Kỹ thuật nhiều phiên bản

- Giữ lại giá trị cũ của dữ liệu khi dữ liệu được cập nhật.
- Khi một giao dịch muốn truy xuất một dữ liệu, một phiên bản (version) thích hợp (giá trị) của dữ liệu được chọn để đảm bảo tính khả tuần tự.
- Ý tưởng: một số hành động Read không được chấp nhận ở các kỹ thuật khác vẫn có thể thực hiện thông qua phiên bản cũ của dữ liệu.

Kỹ thuật nhiều phiên bản

- Bất lợi: Cần phải lưu trữ nhiều dữ liệu hơn tuy nhiên thông thường các dữ liệu cũ vẫn cần được lưu trữ cho các mục đích khác như
 - Phục hồi (recovery)
 - Lịch sử thay đổi của dữ liệu (data history)
- 2 kỹ thuật nhiều phiên bản:
 - Dựa trên thứ tự nhãn thời gian (Timestamp Ordering).
 - Dựa trên khóa 2 pha (2PL).

Kỹ thuật nhiều phiên bản dựa trên nhãn thời gian
(Multiversion Technique
Based on Timestamp Ordering)

KỸ THUẬT NHIỀU PHIÊN BẢN

Multiversion – Timestamp

- Nhiều phiên bản $X_1, X_2, \dots, X_k$ của X được lưu trữ, với mỗi phiên bản X_i :
 - $\text{Read_TS}(X_i)$: giá trị Timestamp lớn nhất của giao dịch đọc thành công X_i .
 - $\text{Write_TS}(X_i)$: giá trị Timestamp của giao dịch ghi X_i .
- Mỗi khi giao dịch T thực hiện $\text{Write}(X)$, một giá trị X_{k+1} của X được tạo ra với cả $\text{Write_TS}(X_{k+1})$ và $\text{Read_TS}(X_{k+1})$ bằng $\text{TS}(T)$.
- Tương tự, mỗi khi giao dịch T thực hiện $\text{Read}(X_i)$, giá trị của $\text{Read_TS}(X_i)$ được gán bằng giá trị lớn hơn (mới hơn) giữa 2 giá trị $\text{Read_TS}(X_i)$ hiện tại và $\text{TS}(T)$.

Multiversion – Timestamp

Để đảm bảo tính khả tuần tự phải thỏa 2 luật sau:

- Nếu T thực hiện Write(X):
 - Nếu **Read_TS(Xi)** > TS(T) thì hủy bỏ và rollback T. Trong đó phiên bản i của X là phiên bản có giá trị Write_TS(Xi) lớn nhất trong tất cả phiên bản của X (nhưng nhỏ hơn hoặc bằng TS(T)).
 - Ngược lại, tạo phiên bản Xj của X với $\text{Read_TS}(X_j) = \text{Write_TS}(X_j) = \text{TS}(T)$

Multiversion – Timestamp

- Nếu T thực hiện Read(X), tìm phiên bản i của X có **Write_TS(Xi)** cao nhất và nhỏ hơn hoặc bằng TS(T), trả về giá trị Xi và thay đổi Read_TS(Xi) bằng giá trị lớn nhất trong 2 giá trị TS(T) và Read_TS(Xi)
- Ta có thể thấy, việc Read(X) luôn thành công.

Multiversion – Timestamp

- Trong trường hợp $\text{Write}(X)$, giao dịch T có thể bị hủy bỏ khi T ghi một phiên bản X mà cần được đọc bởi giao dịch T' (sau T , có timestamp là $\text{Read_TS}(X_i)$) nhưng T' đã đọc phiên bản X_i được ghi bởi giao dịch có timestamp $\text{Write_TS}(X_i)$ (không phải T).

Multiversion – Timestamp

- T: Write(X)
- Các phiên bản của X đang có tại thời điểm đó: $X_1, X_2, \dots, X_i, \dots, X_n$
- i là phiên bản thỏa điều kiện:
 1. $Write_TS(X_i) \leq TS(T)$
 2. $Write_TS(X_i)$ lớn nhất trong số các phiên bản phù hợp điều kiện 1 (sau cùng)
- $Read_TS(X_i) > TS(T)$: rollback

Khóa 2 pha đa phiên bản
(Multiversion 2-phase Locking)

KỸ THUẬT NHIỀU PHIÊN BẢN

Multiversion – 2PL

- Trong mô hình khóa tiêu chuẩn: khóa đọc (read lock/shared lock) và khóa ghi (write lock/exclusive lock)
- Bảng tương thích khóa:
 - Ngang: giao dịch giữ khóa
 - Dọc: giao dịch yêu cầu khóa
- 3 trạng thái khóa của một dữ liệu:
 - Read-locked
 - Write-locked
 - Certify-locked

	Read	Write
Read	yes	no
Write	no	no

Multiversion – 2PL

- Ý tưởng: Cho phép giao dịch T' đọc giá trị X trong khi X đang ở trạng thái khóa ghi bởi giao dịch T.
- Thực hiện: cho phép 2 phiên bản của X
 - Một phiên bản X chỉ được ghi khi giao dịch hoàn tất.
 - Một phiên bản X' được tạo ra khi có giao dịch T giữ khóa ghi trên X.
- Các giao dịch có thể tiếp tục đọc giá trị X.
- Giao dịch T đang giữ khóa trên X có thể tiếp tục ghi các giá trị X' khi cần.

Multiversion – 2PL

- Tuy nhiên, khi T muốn hoàn tất (commit), T phải đặt một khóa certify trên tất cả các giá trị mà T đang giữ khóa ghi.
- Khi đó T phải đợi cho đến khi tất cả các giá trị đó được mở khóa hoàn toàn bởi các giao dịch đang giữ khóa đọc mới có thể hoàn tất việc đặt khóa Certify.
- Cập nhật X bằng X', xóa X' và mở khóa certify.

Multiversion – 2PL

- Bảng tương thích khóa (Lock compatibility table)

	Read	Write	Certify
Read	yes	yes	no
Write	yes	no	no
Certify	no	no	no

Một số kĩ thuật khác

- Giao thức dựa trên tính hợp lệ: Validation (Optimistic) Concurrency Control Techniques.
- Đa hạt: Multiple Granularity Locking

END!

Tham khảo: Chương 23

Fundamentals of Database Systems, 6th Edition